

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Patteela Yashwanth kumar	Reg.No.:	192471055
Department:	CSE Bioscience	Date:	27/08/2026

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required

- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency $< 10 \mu s$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I Patteela Yashwanth Kumar/192471055 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Patteela Yashwanth kumar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Constraint-Based Problem Solving – I/O System Designs

1. I/O Communication Mechanism for a Real-Time Embedded System

Constraints

- CPU utilization must remain below 15%.
- Multiple sensors generate interrupts every 1 ms.
- Vectored interrupt handling is required.
- Memory-mapped and I/O-mapped I/O must be evaluated for latency.

Design

- Use a priority-based vectored interrupt controller with a separate vector for each sensor or sensor group.
- Use DMA to transfer sensor data directly into pre-allocated memory buffers, reducing CPU involvement.
- Keep the interrupt service routine short: acknowledge the device, identify the completed buffer, and schedule deferred processing.
- Select memory-mapped I/O for sensor control and status registers because normal load/store instructions provide low-overhead register access on typical embedded processors.
- Use double buffering so DMA can fill one buffer while the CPU processes the previous buffer.

Constraint Satisfaction

- DMA handles repetitive data movement, helping keep CPU utilization below 15%.
- Vectored interrupts eliminate device polling and reduce interrupt-service time.
- Long computations are moved out of the ISR into lower-priority tasks.
- Memory-mapped I/O is preferred when the processor provides efficient load/store access to device registers.
- Interrupt priorities ensure urgent sensor events are serviced before non-critical tasks.

Conclusion

- The DMA-based, vectored-interrupt architecture provides low latency with low CPU utilization and is suitable for a real-time embedded system.

2. High-Speed NVMe over PCIe I/O Subsystem

Constraints

- Storage throughput must exceed 3 GB/s.

- NVMe over PCIe must be used.
- The CPU should not handle bulk data transfer.
- A DMA controller with interrupt notification is required.

Design

- Connect the NVMe storage device to the processor through a high-speed PCIe interface with sufficient lanes and link bandwidth.
- Use NVMe submission and completion queues stored in system memory.
- Use PCIe DMA so storage data moves directly between the NVMe device and main memory without CPU copying.
- Use MSI/MSI-X interrupt notification for efficient completion signaling.
- Use multiple queue pairs and sufficient queue depth to support parallel I/O operations.

Constraint Satisfaction

- A suitable PCIe link provides enough bandwidth to sustain more than 3 GB/s.
- NVMe supports parallel queues and efficient command processing for solid-state storage.
- DMA performs bulk data movement while the CPU mainly prepares commands and handles completions.
- Interrupts notify the CPU after completed operations instead of requiring continuous polling.
- Aligned buffers and large I/O requests help maintain high throughput.

Conclusion

- NVMe over PCIe with DMA, multiple queues, and MSI/MSI-X notification can achieve high storage throughput while keeping CPU involvement low.

3. Multi-Device Communication Architecture

Constraints

- Six peripherals use USB.
- Two high-speed devices use Thunderbolt.
- Interrupt starvation must be avoided.
- Both hardware and software interrupts must be used.
- Fair bus arbitration is required.

Design

- Connect the six USB peripherals through USB host controllers or hubs with appropriate bandwidth allocation.

- Connect the two high-speed devices through a Thunderbolt controller using highspeed paths.
- Use hardware interrupts for urgent events such as completed transfers, errors, and device connection changes.
- Use software interrupts or deferred processing for lower-priority protocol and background tasks.
- Use round-robin or weighted-fair arbitration so no device can continuously monopolize shared bus resources.

Starvation Prevention

- Assign higher priority to time-sensitive interrupts while keeping every ISR short.
- Batch low-priority notifications to reduce excessive interrupt frequency.
- Use short critical sections and avoid unnecessary interrupt masking.
- Give each bus client a bounded service opportunity using time slices, transaction quotas, or weighted scheduling.
- Monitor queue delays so continuously active USB devices cannot starve other USB or Thunderbolt devices.

Conclusion

- The combination of hardware interrupts, software interrupts, priority control, and fair arbitration provides balanced and responsive multi-device communication.

4. Data Acquisition System at 500 MB/s

Constraints

- Continuous data stream is 500 MB/s.
- Minimal CPU intervention is required.
- DMA must be compared with interrupt-driven I/O.
- Device registers must use memory-mapped I/O.

Design

- Map device control and status registers into the processor address space using memory-mapped I/O.
- Use DMA to transfer acquired samples directly from the device into a circular buffer in system memory.
- Use double or ring buffering so acquisition continues while software processes completed buffers.

- Generate interrupts when a buffer reaches a threshold instead of interrupting for every sample.
- Process completed buffers in a background task while DMA continues acquisition.

DMA vs Interrupt-Driven I/O

- DMA is preferred because a 500 MB/s stream would create excessive CPU overhead if data were transferred by individual interrupts.
- Interrupt-driven I/O is simpler and suitable for low-rate devices, but frequent interrupts at this data rate can consume substantial CPU time.
- DMA transfers large blocks with little CPU intervention and supports sustained high throughput.
- Interrupts remain useful for DMA completion, buffer availability, and error notification.
- Memory-mapped registers provide direct and low-overhead access to device control and status information.

Conclusion

- For a continuous 500 MB/s stream, DMA with memory-mapped device registers is the appropriate solution because the CPU handles control and processing rather than bulk data movement.

5. Interrupt Handling Mechanism with Nested Interrupts

Constraints

- Nested interrupts must be supported.
- Maximum interrupt latency must be below 10 μ s.
- Vectored interrupts are required for high-priority devices.
- Software interrupts must handle background tasks.

Design

- Use a programmable interrupt controller with multiple priority levels and individual vectors for high-priority devices.
- Enable nested interrupts so a higher-priority interrupt can preempt a lower-priority ISR.
- Keep high-priority ISRs extremely short: save essential context, acknowledge the device, record the event, and defer lengthy processing.
- Use software interrupts for background and deferred processing so non-critical work does not block urgent hardware interrupts.

- Use fixed priorities, bounded critical sections, fast context switching, and DMA where appropriate to meet the sub-10- μ s latency target.

Latency Control

- Keep the interrupt vector table in fast-access memory when supported by the processor.
- Avoid long loops, blocking operations, and complex calculations inside ISRs.
- Mask interrupts only for the shortest necessary critical sections.
- Use DMA for large transfers so ISRs do not spend time copying data.
- Measure worst-case interrupt response using hardware timers or tracing and verify that the maximum remains below 10 μ s.

Conclusion

- A priority-based nested interrupt architecture with vectored high-priority interrupts and software interrupts for deferred work can satisfy the required latency when implemented on suitable real-time hardware.